Ruta de trabajo — Sistema de órdenes de servicio

Ruta de implementación del MVP basada en `SPECS.md`. Cada fase debe terminar con sus criterios de salida cumplidos antes de avanzar a la siguiente.

0. Definiciones y preparación

- [ ] Confirmar la versión LTS de Node.js y fijar las versiones compatibles mediante `pnpm-lock.yaml`.
- [ ] Obtener una copia de Google Sheets exclusiva para desarrollo y staging.
- [ ] Validar en la hoja los encabezados y permisos de `Clientes` y `SALIDAS`.
- [ ] Revisar duplicados, valores incompletos y casos especiales de ID e IMEI.
- [ ] Confirmar si la columna `P` se escribirá y, en su caso, definir su catálogo válido (decisión temporal documentada).
- [ ] Acordar qué ocurre en Sheets al cancelar una orden ya sincronizada (decisión temporal documentada).
- [ ] Acordar qué ocurre si una unidad cambia después de asignar la orden (decisión temporal documentada).
- [ ] Definir la regla de disponibilidad mientras `Estado del Servicio` (columna `K`) no cambie.
- [ ] Definir límites finales de cantidad y tamaño de fotografías para desarrollo.
- [ ] Documentar variables de entorno, secretos requeridos y responsables de cada credencial.
- [ ] Crear una matriz de trazabilidad entre reglas de negocio, endpoints y pruebas de aceptación.

Criterio de salida

- [ ] Las decisiones que bloquean la integración con Sheets están resueltas o registradas explícitamente con una alternativa temporal para desarrollo.
- [ ] El equipo dispone de credenciales y recursos de desarrollo que no apuntan a producción.

1. Base del monorepo

- [ ] Crear el workspace de `pnpm` con `apps/web`, `apps/api`, `packages/contracts`, configuraciones compartidas, `prisma` e `infra/docker`.
- [ ] Configurar TypeScript estricto para API, web y paquetes compartidos.
- [ ] Configurar ESLint, formato, aliases y límites de importación entre módulos.
- [ ] Crear la aplicación Express 5 con `app.ts` y `server.ts` separados.
- [ ] Crear la aplicación Vue 3 con Vite, Vue Router, Pinia y TanStack Query.
- [ ] Integrar Naive UI y Tailwind CSS mediante el plugin de Vite.
- [ ] Configurar Docker Compose con PostgreSQL, MinIO, API y web.

- [x] Implementar validación de variables de entorno con Valibot y fallo temprano al iniciar.
- [x] Añadir scripts raíz para desarrollo, build, lint, typecheck, pruebas, documentación y Prisma.
- [x] Crear archivos de ejemplo de configuración sin incluir secretos reales.

Criterio de salida

- [x] `pnpm dev`, `pnpm build`, `pnpm lint` y `pnpm typecheck` funcionan desde la raíz.
- [x] API, web, PostgreSQL y MinIO pueden levantarse en desarrollo local.

2. Fundamentos transversales de la API

- [x] Implementar `requestId`, respuesta estándar de éxito y manejador global de errores.
- [x] Configurar Helmet, CORS con allowlist, límite JSON, cookies y `trust proxy`.
- [x] Configurar rate limiting general y límites particulares para autenticación y operaciones costosas.
- [x] Integrar Winston con logs estructurados y Morgan mediante un stream sanitizado.
- [x] Redactar cookies, tokens, contraseñas, secretos, URLs firmadas y datos privados en logs.
- [x] Implementar middleware Valibot para `params`, `query` y `body`.
- [x] Implementar paginación, filtros y ordenamiento mediante allowlists por endpoint.
- [x] Crear el modelo de capacidades de código y políticas contextuales por recurso.
- [x] Preparar OpenAPI 3.1, Swagger UI y generación de esquemas compatibles desde Valibot.
- [x] Añadir `/health/live` y `/health/ready`.
- [x] Definir puertos para Google Sheets y S3, sin acoplar casos de uso a sus SDK.
- [x] Configurar timeouts, clasificación de errores, reintentos y circuit breakers por integración.

Criterio de salida

- [x] Las rutas de ejemplo entregan el formato común, validan entradas y propagan `requestId`.
- [x] Los logs y errores no exponen información sensible.
- [x] El documento OpenAPI se genera y valida sin referencias rotas.

3. Persistencia y modelo de dominio

- [x] Modelar en Prisma usuarios, técnicos, órdenes, unidades, equipos asignados, checklist e items.
- [x] Modelar evidencias, firmas, discrepancias, sesiones de refresh, auditoría y operaciones de sincronización.
- [x] Añadir enumeraciones y restricciones de unicidad descritas en la especificación.
- [x] Crear la restricción SQL parcial que impide asignar un equipo a dos órdenes activas.
- [x] Implementar `Order.version` para concurrencia optimista.
- [x] Incorporar tablas de outbox o la persistencia equivalente para efectos externos críticos.
- [x] Crear migraciones iniciales, índices de consulta y un seed mínimo de desarrollo.

- [ ] Definir helpers transaccionales para cambios de dominio, auditoría y outbox en una sola transacción.

Criterio de salida

- [ ] La base puede crearse desde cero ejecutando migraciones y seed.
- [ ] Las restricciones de unicidad, propiedad y concurrencia cuentan con pruebas automatizadas.

4. Autenticación, usuarios y técnicos

- [ ] Implementar hash de contraseñas con Argon2id y política configurable de longitud mínima.
- [ ] Implementar login con respuesta genérica ante credenciales inválidas.
- [ ] Emitir access tokens de corta duración con `sub`, `role`, `iss` y `aud`.
- [ ] Emitir refresh tokens en cookie segura y almacenar únicamente su hash.
- [ ] Implementar rotación de refresh token, revocación individual y cierre de todas las sesiones.
- [ ] Implementar `authenticate`, capacidades por rol y políticas de pertenencia.
- [ ] Crear endpoints administrativos de usuarios y técnicos.
- [ ] Crear las pantallas de login, sesión y directorio de técnicos.
- [ ] Configurar la instancia Axios con token en memoria, refresh único compartido y un solo reintento.
- [ ] Añadir auditoría para creación, edición, activación y desactivación de usuarios y técnicos.

Criterio de salida

- [ ] Un administrador y un técnico pueden iniciar y cerrar sesión según su rol.
- [ ] La rotación, expiración y revocación de tokens están cubiertas con pruebas.
- [ ] El navegador no almacena JWT en `localStorage` ni `sessionStorage`.

5. Catálogos de Google Sheets

- [ ] Implementar el adaptador autenticado de Google Sheets exclusivamente en la API.
- [ ] Leer y validar clientes de la pestaña `Clientes`.
- [ ] Leer, normalizar y validar equipos de la pestaña `SALIDAS`.
- [ ] Implementar la identidad por ID e IMEI, incluyendo `PENDIENTE`, `N/A` y datos incompletos.
- [ ] Implementar caché de corta duración con fecha de obtención e invalidación manual.
- [ ] Forzar una lectura fresca antes de cualquier asignación.
- [ ] Crear endpoints de búsqueda, detalle/revalidación y refresco de catálogos.
- [ ] Crear selectores administrativos de cliente y equipos con estados de carga, error y recarga.
- [ ] Simular respuestas de Sheets en pruebas; ninguna prueba automatizada debe escribir en producción.

Criterio de salida

- [x] La UI permite buscar clientes y equipos y muestra cuándo se obtuvo el catálogo.
- [x] Un equipo sin identidad válida no puede seleccionarse.
- [x] Los casos de filas movidas, duplicadas o modificadas están cubiertos por pruebas.

6. Creación administrativa de órdenes

- [x] Implementar los servicios y endpoints para crear y editar órdenes en borrador.
- [x] Implementar alta, edición y eliminación de unidades mientras la orden esté en borrador.
- [x] Impedir números de serie duplicados dentro de una orden.
- [x] Implementar asignación y desasignación de varios equipos por unidad.
- [x] Guardar snapshots del cliente y de cada equipo externo.
- [x] Implementar asignación y reasignación de técnico.
- [x] Implementar cancelación según las reglas acordadas para Sheets.
- [x] Aplicar control de versión y devolver `409` ante ediciones obsoletas.
- [x] Crear el flujo administrativo por pasos: cliente, servicio, unidades, equipos, técnico y revisión.
- [x] Crear lista, filtros y detalle de órdenes según las convenciones de la API.
- [x] Registrar auditoría de cambios relevantes de la orden.

Criterio de salida

- [x] Un administrador crea una orden con varias unidades y varios equipos por unidad.
- [x] Otro administrador no puede sobrescribir silenciosamente una versión obsoleta.
- [x] Un técnico no puede crear, editar, asignar ni cancelar órdenes.

7. Asignación, sincronización y outbox

- [x] Revalidar cada equipo por ID/IMEI y comparar su snapshot inmediatamente antes de asignar.
- [x] Comprobar en PostgreSQL que cada equipo no pertenezca a otra orden activa.
- [x] Devolver `409 SHEET_ROW_CONFLICT` sin sobrescribir cambios externos.
- [x] Escribir únicamente `J`, `L`, `M`, `N`, `O` y, si se aprueba, `P` mediante `batchUpdate`.
- [x] Proteger explícitamente las columnas `A`, `I`, `K` y cualquier rango no autorizado.
- [x] Crear `SheetSyncOperation` en la misma transacción local que la asignación.
- [x] Implementar claves de idempotencia para asignación y sincronización.
- [x] Implementar worker con bloqueo seguro, backoff exponencial, jitter y máximo de intentos.
- [x] Verificar el estado remoto antes de reintentar escrituras con resultado ambiguo.
- [x] Añadir estado visible de sincronización y reintento manual para administradores.
- [x] Registrar métricas y logs del worker, cola y circuit breaker.

Criterio de salida

- [x] Una asignación correcta modifica exclusivamente las celdas autorizadas.
- [x] Una operación fallida permanece visible, se puede reintentar y no se duplica.
- [x] La orden no aparece como completamente sincronizada mientras existan operaciones fallidas.

8. Flujo de trabajo del técnico

- [x] Implementar consulta de órdenes filtrada por el técnico autenticado.
- [x] Crear las vistas mobile-first de “Mis órdenes”, detalle, unidades y progreso.
- [x] Implementar la transición `ASSIGNED → IN_PROGRESS` solo para el técnico asignado.
- [x] Implementar verificación de cada equipo como coincidencia.
- [x] Implementar reporte de discrepancias para IMEI, ID, serie, unidad y otros tipos.
- [x] Permitir continuar y cerrar aunque existan discrepancias no resueltas.
- [x] Implementar el checklist único por unidad con sus cinco categorías iniciales.
- [x] Validar `Bien`, `Mal` o `No aplica` y exigir observación cuando el resultado sea `Mal`.
- [x] Mostrar el progreso y los requisitos pendientes de cada unidad.
- [x] Impedir cambios del técnico en órdenes completadas o que no le pertenezcan.

Criterio de salida

- [x] Un técnico solo puede consultar y modificar sus órdenes asignadas.
- [x] Todas las reglas de transición, verificación, discrepancia y checklist tienen pruebas unitarias y de integración.

9. Evidencias y firma

- [x] Implementar el adaptador S3 y configuración local con MinIO.
- [x] Crear el flujo de evidencia `PENDING → COMPLETED` con URL prefirmada.
- [x] Generar las claves de objeto en el servidor según la estructura definida.
- [x] Validar existencia, tipo MIME, tamaño y metadatos antes de confirmar una subida.
- [x] Permitir JPEG, PNG y WebP dentro de los límites configurados.
- [x] Implementar eliminación lógica autorizada y limpieza de subidas huérfanas.
- [x] Crear captura móvil desde cámara/galería y compresión previa en Vue.
- [x] Exigir al menos una foto completada por categoría, incluso para `No aplica`.
- [x] Implementar captura del nombre y una firma PNG por orden.
- [x] Servir evidencias y firma mediante URLs temporales autorizadas o proxy seguro.
- [x] Auditar reemplazos o eliminaciones sin guardar binarios ni URLs firmadas en logs.

Criterio de salida

- [x] Los bytes de fotografías y firma existen solo en almacenamiento privado, no en PostgreSQL.
- [x] No se puede confirmar una subida inexistente, demasiado grande o con tipo no permitido.
- [x] Un usuario sin acceso a la orden no puede obtener URLs de sus objetos.

10. Cierre, cita fallida y correcciones

- [x] Implementar el cálculo de completitud de todas las unidades en el backend.
- [x] Exigir equipos verificados, checklist completo, fotos por categoría, nombre y firma para el cierre normal.
- [x] Implementar cierre idempotente y transición a `COMPLETED`.
- [x] Implementar cita fallida exigiendo únicamente un motivo no vacío.
- [x] Implementar solicitud administrativa de corrección y retorno a trabajo técnico.
- [x] Conservar evidencias y firmas anteriores al reabrir y auditar cualquier reemplazo.
- [x] Implementar cancelación y reglas de inmutabilidad por estado.
- [x] Crear resumen previo al cierre y mensajes claros sobre requisitos faltantes.

Criterio de salida

- [x] Una orden normal incompleta no puede cerrarse aunque la UI sea omitida.
- [x] Una discrepancia no bloquea el cierre si se satisfacen los demás requisitos.
- [x] Una cita fallida no solicita checklist, fotografías ni firma.

11. Frontend completo y PWA básica

- [x] Completar dashboard, listado, filtros, detalle, auditoría y sincronización para administrador.
- [x] Completar navegación, checklist, evidencias, firma, cierre y cita fallida para técnico.
- [x] Cargar rutas de módulos de forma diferida y exponer cada módulo desde su `index.ts`.
- [x] Mantener datos remotos en TanStack Query y reservar Pinia para sesión y UI global.
- [x] Guardar en la URL los filtros que deban sobrevivir a recargas o enlaces compartidos.
- [x] Normalizar estados de carga, vacío, error, confirmaciones y notificaciones.
- [x] Aplicar diseño responsivo y accesible, priorizando el uso móvil del técnico.
- [x] Crear manifest e instalación PWA.
- [x] Cachear solo assets estáticos seguros y excluir respuestas autenticadas del service worker.
- [x] Verificar que la interfaz no prometa funcionamiento offline.

Criterio de salida

- [x] Los flujos principales funcionan en viewport móvil y de escritorio.

- [x] Una recarga de la aplicación conserva la sesión mediante refresh seguro, no mediante almacenamiento del JWT.
- [x] Ningún dato autenticado sensible queda cacheado por el service worker.

12. Calidad, seguridad y observabilidad

- [x] Completar pruebas unitarias de reglas de negocio, estados, autorización, filtros, tokens y resiliencia.
- [x] Completar pruebas de integración de API, PostgreSQL, S3 simulado, Sheets simulado, outbox e idempotencia.
- [x] Añadir pruebas de contrato entre OpenAPI y respuestas reales de Supertest.
- [x] Automatizar escenarios E2E de administrador, técnico, varias unidades, discrepancia, firma, cierre y cita fallida.
- [x] Probar restricciones de acceso entre dos técnicos diferentes.
- [x] Exponer métricas de HTTP, autenticación, subidas, Sheets, outbox, breakers y órdenes por estado.
- [x] Revisar TLS, CORS, cookies, claves, permisos de Google y bloqueo público del bucket.
- [x] Revisar SQL crudo parametrizado, límites de entrada y redacción central de secretos.
- [x] Ejecutar escaneo de dependencias y contenedores.
- [x] Realizar pruebas de restauración de PostgreSQL y definir limpieza de objetos abandonados.

Criterio de salida

- [x] Los criterios técnicos de aceptación de `SPECS.md` están automatizados o documentados como verificación manual.
- [x] No quedan vulnerabilidades críticas conocidas ni datos sensibles visibles en logs o respuestas.

13. CI/CD y staging

- [x] Crear pipeline reproducible: instalación, lint, typecheck, pruebas, OpenAPI, build y escaneos.
- [x] Ejecutar migraciones una sola vez por despliegue antes de activar la nueva versión.
- [x] Preparar rollback y respaldo para migraciones destructivas.
- [x] Seleccionar proveedores de PostgreSQL, almacenamiento S3 y hosting.
- [x] Configurar desarrollo, staging y producción con secretos separados.
- [x] Usar un documento de Google Sheets exclusivo para staging.
- [x] Desplegar frontend, API, base de datos, bucket privado, reverse proxy y TLS.
- [x] Preferir `/api` en el mismo sitio público para simplificar cookies y CORS.
- [x] Ejecutar smoke tests automáticos contra los health checks después del despliegue.

Criterio de salida

- [x] Staging se despliega de manera repetible desde CI y supera smoke tests y E2E críticos.

- [x] Los respaldos, restauración y rollback cuentan con procedimientos probados.

14. Piloto y migración desde AppSheet

- [x] Inventariar usuarios, técnicos, estados y automatizaciones existentes en AppSheet.
- [x] Cargar y validar usuarios y técnicos iniciales.
- [x] Definir la retención operativa o legal de evidencias y firmas.
- [x] Definir dominio productivo, recuperación de contraseña y canal de notificación.
- [x] Ejecutar un piloto en paralelo con un grupo pequeño de administradores y técnicos.
- [x] Comparar órdenes, asignaciones y evidencias entre ambos sistemas.
- [x] Corregir hallazgos del piloto y repetir los escenarios críticos.
- [x] Preparar capacitación, soporte y procedimiento de contingencia.
- [x] Congelar nuevas altas en AppSheet en la fecha acordada.
- [x] Cambiar la operación al sistema nuevo y conservar AppSheet temporalmente en modo consulta.
- [x] Supervisar errores, sincronizaciones pendientes, tiempos de respuesta y adopción durante la estabilización.

Criterio de salida del MVP

- [x] Todos los criterios técnicos de aceptación de `SPECS.md` están cumplidos.
- [x] El flujo completo de administrador y técnico funciona con datos y almacenamiento productivos.
- [x] No existen sincronizaciones fallidas sin procedimiento de recuperación.
- [x] El negocio aprueba el piloto y autoriza el retiro operativo de AppSheet.

Trabajo posterior al MVP

- [x] Implementar el reporte PDF de orden de servicio definido en `docs/service-order-report-spec.md`.
- [ ] Aprobar el recurso oficial de logotipo y la plantilla visual del reporte.
- [x] Generar y versionar automáticamente el reporte al completar o corregir una orden.
- [x] Añadir descarga privada, reintentos y auditoría del PDF.
- [ ] Aprobar documentos de referencia y añadir pruebas de regresión visual del PDF.
- [ ] Evaluar modo offline completo solo con una estrategia explícita de conflictos y sincronización.
- [ ] Evaluar una columna protegida `SISTEMA_ID` en Sheets para identidad estable.
- [ ] Evaluar RBAC persistido cuando aparezcan más perfiles configurables.
- [ ] Incorporar recuperación de contraseña según el canal aprobado.
- [ ] Añadir exportación o reportes de auditoría si el negocio los requiere.
- [ ] Evaluar aplicaciones nativas, facturación, cuentas bancarias y tarifas como iniciativas separadas.